



Politechnika Łódzka



IADPŁ

INSTYTUT AUTOMATYKI
POLITECHNIKI ŁÓDZKIEJ

Zakład sterowania robotów

ROBOTY MOBILNE

Instrukcja laboratoryjna 2

Podstawy Python'a z uwzględnieniem operacji macierzowych
i ROS 2



23 października 2025

Spis treści

1	Podstawy języka Python	2
1.1	Cechy ogólne	2
1.2	Importy	2
1.2.1	Podstawowe sposoby importu	2
1.3	Zmienne i typy danych	2
1.3.1	Konwersja typów	3
1.3.2	Operacje na tekście (stringach)	3
1.3.3	Łączenie zmiennych i tekstu	3
1.4	Operatory	4
1.4.1	Operatory arytmetyczne	4
1.4.2	Operatory porównania	4
1.4.3	Operatory logiczne	5
1.4.4	Operatory przynależności	5
1.4.5	Operatory identyczności	5
1.5	Struktury danych	6
1.5.1	Lista (list)	6
1.5.2	Słownik (dict)	6
1.5.3	Krotka (tuple)	6
1.5.4	Zbiór (set)	7
1.6	Instrukcje warunkowe i pętle	7
1.6.1	Instrukcje warunkowe if/elif/else	7
1.6.2	Pętla for	7
1.6.3	Pętla while	8
1.6.4	Dodatkowe instrukcje w pętlach	8
1.7	Funkcje	9
1.7.1	Definicja funkcji	9
1.7.2	Argumenty funkcji	9
1.8	Klasy	9
1.8.1	Definicja klasy	9
1.8.2	Atrybuty i metody	10
1.8.3	Dziedziczenie	10
2	Operacje matematyczne	10
2.1	Przydatne funkcje matematyczne	10
2.1.1	Stałe i podstawowe funkcje	11
2.1.2	Funkcje trygonometryczne	11
2.1.3	Funkcja np.arctan2	11
2.2	Wektory i macierze	12

2.2.1	Operacje na macierzach	13
2.2.2	Konwersja danych NumPy na listy Pythona	14
2.2.3	Macierz odwrotna i pseudoinwersja	15
3	ROS 2 i Python	16
3.1	Przykładowe importy w Pythonie dla ROS 2	17
3.2	Definiowanie noda w Pythonie	17
3.3	Callbacki – funkcje wywoływane automatycznie	18
3.4	Konwersja kąta na kwaternion	18
3.5	Funkcja main w Pythonie	18

1 Podstawy języka Python

1.1 Cechy ogólne

- Python jest językiem interpretowanym – kod wykonywany jest sekwencyjnie tzn. linia po linii.
- Bloki kodu wyznaczane są **wcięciami**, a nie klamrami {}.

1.2 Importy

Python posiada ogromną liczbę wbudowanych i zewnętrznych bibliotek. Aby skorzystać z ich funkcji, należy je najpierw **zaimportować**. Służy do tego słowo kluczowe `import`.

1.2.1 Podstawowe sposoby importu

- `import math` – import całej biblioteki. Dostęp do funkcji odbywa się przez nazwę pakietu, np. `math.sqrt(9)`.
- `import numpy as np` – import z aliasem (skrótowa nazwa). W praktyce dla pakietu **NumPy** prawie zawsze stosuje się alias `np`.
- `from geometry_msgs.msg import Twist` – import tylko wybranego elementu z biblioteki. W tym przykładzie z pakietu ROS 2 wczytywana jest wiadomość `Twist`.

```
1 import math
2 import numpy as np      # alias, zalecane dla numpy
3 from geometry_msgs.msg import Twist
4
5 print(math.sqrt(16))    # 4.0
6 print(np.pi)           # 3.141592653589793
```

Kod źródłowy 1: Różne sposoby importowania bibliotek w Pythonie

Uwagi praktyczne

- Importy zazwyczaj umieszcza się na początku pliku Pythona.
- Można importować wiele elementów z jednej biblioteki: `from math import sin, cos, pi`
- Można nadać własny alias zmienionej nazwy: `from math import sqrt as pierwiastek`
- W kontekście ROS 2 często importujemy wiadomości z pakietów `geometry_msgs`, `sensor_msgs` czy `nav_msgs`.

1.3 Zmienne i typy danych

- W odróżnieniu od języków takich jak C czy Java, **nie trzeba podawać typu zmiennej przy jej deklaracji**. Zmienne w Pythonie tworzone są automatycznie w momencie przypisania im wartości.
- Python sam rozpoznaje, z jakim typem danych ma do czynienia.
- Python rozróżnia wielkość liter – zmienne `A` i `a` to różne obiekty.
- Język jest **dynamicznie typowany**, co oznacza, że ta sama zmienna może w trakcie działania programu przyjąć różne typy wartości.
- W odróżnieniu od C/C++, w Pythonie nie trzeba rezerwować pamięci ani definiować rozmiaru zmiennej.

- Nadawanie czytelnych nazw zmiennym jest ważne, ponieważ brak deklaracji typów utrudnia czytelność kodu. W wyższych wersjach Pythona możliwe jest wstawienie typowania. Bardziej zaawansowane edytory kodu potrafią zweryfikować niezgodność typów.

```
1 a = 5          # int - liczba całkowita
2 b = 3.14       # float - liczba zmiennoprzecinkowa
3 text = "Hello" # string - napis
4 flag = True    # bool - wartość logiczna
```

Kod źródłowy 2: Zmienne i podstawowe typy danych

Funkcja `type(a)` zwraca typ zmiennej, np. `<class 'int'>`.

1.3.1 Konwersja typów

Często potrzebna jest zmiana typu zmiennej, np. z napisu na liczbę:

```
1 int("5")      # zamiana tekstu na liczbę całkowitą -> 5
2 float(3)      # zamiana int na float -> 3.0
3 str(5)        # zamiana liczby na napis -> "5"
4 bool(0)       # wynik: False
5 bool(1)       # wynik: True
```

Kod źródłowy 3: Konwersje typów w Pythonie

1.3.2 Operacje na tekście (stringach)

Tekst w Pythonie to sekwencje znaków. Można je indeksować, dzielić na fragmenty (ang. *slicing*) oraz modyfikować za pomocą metod wbudowanych:

```
1 s = "Python"
2 print(len(s))    # długość tekstu -> 6
3 print(s[0])      # pierwszy znak -> 'P'
4 print(s[-1])     # ostatni znak -> 'n'
5 print(s[0:3])    # fragment od 0 do 2 -> 'Pyt'
6 print(s.lower()) # 'python'
7 print(s.upper()) # 'PYTHON'
```

Kod źródłowy 4: Podstawowe operacje na tekście

1.3.3 Łączenie zmiennych i tekstu

W praktyce często zachodzi potrzeba wyświetlenia informacji o aktualnych wartościach zmiennych (np. w logach). Python udostępnia kilka sposobów na łączenie tekstu i zmiennych:

```
1 x = 5
2 y = 3.14
3 name = "Robot"
4
5 # 1. Prosta konkatencja (tylko dla stringów)
6 print("x = " + str(x) + ", y = " + str(y))
7
8 # 2. Metoda format()
9 print("Wartość x = {}, y = {}".format(x, y))
10
11 # 3. F-string (Python 3.6+, najbardziej czytelne rozwiązanie) - zmienna w tekście wstawiana jest pomiędzy
   ↪ nawiasami klamrowymi {} np. {x}, {y}, {name}
12 print(f"Wartość zmiennej x = {x}, zmiennej y = {y}")
13 print(f"Nazwa obiektu: {name}")
```

Kod źródłowy 5: Formatowanie tekstu z wartościami zmiennych

Wynik działania programu Kod źródłowy 5 w konsoli:

```
1 x = 5, y = 3.14
2 Wartość x = 5, y = 3.14
3 Wartość zmiennej x = 5, zmiennej y = 3.14
4 Nazwa obiektu: Robot
```

Kod źródłowy 6: Wynik działania programu w konsoli

Uwagi praktyczne

- Python nie posiada typu `char` – każdy pojedynczy znak jest tekstem o długości jeden.
- Metoda `str(x)` zamienia dowolny obiekt na tekst – dzięki temu można go połączyć z innymi stringami.
- F-stringi pozwalają wstawić zmienną *bezpośrednio* wewnątrz tekstu i są najbardziej przejrzyste.
- W f-stringach można wykonywać obliczenia w miejscu wstawienia, np.:

```
1 a = 10
2 b = 3
3 print(f"Wynik dzielenia {a}/{b} = {a/b:.2f}") # wynik zaokrąglony do 2 miejsc po przecinku
```

Kod źródłowy 7: Obliczenia i formatowanie w f-stringach

Wynik działania programu Kod źródłowy 7 w konsoli:

```
1 Wynik dzielenia 10/3 = 3.33
```

Kod źródłowy 8: Wynik działania programu w konsoli

1.4 Operatory

Operatory w Pythonie działają podobnie jak w innych językach programowania. Wyniki działania operatorów można sprawdzić przy pomocy funkcji `print`.

1.4.1 Operatory arytmetyczne

- `+` dodawanie, `-` odejmowanie, `*` mnożenie, `/` dzielenie (zwraca wynik zmiennoprzecinkowy),
- `%` reszta z dzielenia,
- `**` potęgowanie,
- `//` dzielenie bez reszty (zaokrąglenie w dół).

```
1 print(5 + 2)    # 7
2 print(5 - 2)    # 3
3 print(5 * 2)    # 10
4 print(5 / 2)    # 2.5
5 print(5 % 2)    # 1
6 print(5 ** 2)   # 25
7 print(5 // 2)   # 2
```

Kod źródłowy 9: Operatory arytmetyczne w Pythonie

1.4.2 Operatory porównania

- Wynikiem jest wartość logiczna `True` lub `False`.
- Porównania można łączyć w wyrażeniach logicznych.

```
1 x = 5
2 y = 3
3 print(x == y)    # False (czy x równe y?)
4 print(x != y)    # True  (czy x różne od y?)
5 print(x > y)     # True
6 print(x < y)     # False
7 print(x >= 5)    # True
8 print(y <= 2)    # False
```

Kod źródłowy 10: Operatory porównania

1.4.3 Operatory logiczne

- `and` – koniunkcja (logiczne „i”),
- `or` – alternatywa (logiczne „lub”),
- `not` – negacja.

```
1 a = True
2 b = False
3 print(a and b)    # False
4 print(a or b)     # True
5 print(not a)      # False
```

Kod źródłowy 11: Operatory logiczne

1.4.4 Operatory przynależności

- Służą do sprawdzania, czy element znajduje się w kolekcji (np. liście, napisie).
- `in` – zwraca `True`, jeśli element występuje,
- `not in` – zwraca `True`, jeśli elementu nie ma w kolekcji.

```
1 lista = [1, 2, 3, 4]
2 print(2 in lista)    # True
3 print(5 in lista)    # False
4 print(5 not in lista) # True
5
6 text = "Python"
7 print("P" in text)    # True
8 print("py" in text)   # False (Python rozróżnia wielkość liter)
```

Kod źródłowy 12: Operatory przynależności

1.4.5 Operatory identyczności

Operatory identyczności służą do sprawdzania, czy dwie zmienne odnoszą się do **tego samego obiektu w pamięci**, a nie tylko czy mają taką samą wartość. Różni się to od operatora porównania `==`, który sprawdza równość wartości.

- `is` – zwraca `True`, jeśli obie zmienne wskazują na ten sam obiekt,
- `is not` – zwraca `True`, jeśli zmienne wskazują na różne obiekty.

```
1 a = [1, 2, 3]
2 b = a                # obie zmienne wskazują na tę samą listę
3 c = [1, 2, 3]        # nowa lista o tej samej wartości
4
5 print(a == b)         # True  (wartości takie same)
6 print(a is b)         # True  (ten sam obiekt w pamięci)
7
```

```
8 print(a == c)      # True (wartości takie same)
9 print(a is c)      # False (inne obiekty w pamięci)
10
11 # sprawdzenie negacji
12 print(a is not c) # True
```

Kod źródłowy 13: Operatory identyczności `is` i `is not`

1.5 Struktury danych

Python udostępnia kilka podstawowych struktur danych, które są bardzo elastyczne i często wykorzystywane w praktyce. Najważniejsze z nich to:

- Lista – najczęściej stosowana struktura do przechowywania sekwencji elementów.
- Słownik – przydatny, gdy potrzebny jest szybki dostęp do wartości po nazwie (kluczu).
- Krotka – chroni dane przed przypadkową modyfikacją.
- Zbiór – używany do przechowywania unikalnych wartości oraz do operacji na zbiorach.

Różnią się one przeznaczeniem i sposobem dostępu do elementów.

1.5.1 Lista (list)

Lista to dynamiczna tablica, która może przechowywać elementy różnych typów. Elementy mają przypisane indeksy zaczynające się od 0. Listy można modyfikować – dodawać, usuwać i zmieniać ich elementy.

```
1 L = [1, 2, 3]      # utworzenie listy
2 print(L[0])        # pierwszy element -> 1
3 L.append(4)         # dodanie elementu na końcu
4 L[1] = 10          # zmiana drugiego elementu
5 print(L)           # [1, 10, 3, 4]
```

Kod źródłowy 14: Lista w Pythonie – dynamiczna tablica

1.5.2 Słownik (dict)

Słownik przechowuje dane w postaci **klucz: wartość**. Klucze muszą być unikalne (np. napisy, liczby), a wartości mogą być dowolnym typem.

```
1 D = {"a": 1, "b": 2}
2 print(D["a"])      # dostęp do wartości po kluczu -> 1
3
4 D["c"] = 3         # dodanie nowej pary klucz: wartość
5 D["a"] = 5         # nadpisanie istniejącej wartości dla klucza 'a'
6 print(D)           # {'a': 5, 'b': 2, 'c': 3}
```

Kod źródłowy 15: Słownik w Pythonie – struktura klucz: wartość

1.5.3 Krotka (tuple)

Krotka wygląda podobnie do listy, ale jest **niemodyfikowalna** (ang. *immutable*). Raz utworzonej krotki nie można zmieniać – służy do przechowywania stałych zestawów danych.

```
1 T = (1, 2, 3)
2 print(T[0])        # pierwszy element -> 1
3 T[0] = 10          # błąd: krotka jest niemodyfikowalna -> TypeError: 'tuple' object does not support item
  ↪ assignment
```

Kod źródłowy 16: Krotka – stała lista

1.5.4 Zbiór (set)

Zbiór przechowuje **unikalne** elementy w nieuporządkowanej kolejności. Przydaje się do operacji matematycznych na zbiorach, takich jak suma, część wspólna, różnica.

```
1 S = {1, 2, 3, 3}
2 print(S)           # {1, 2, 3} -- duplikaty są usuwane
3
4 S.add(4)           # dodanie elementu
5 S.remove(2)        # usunięcie elementu
6 print(3 in S)      # True (operator przynależności; 3 znajduje się w zbiorze S)
```

Kod źródłowy 17: Zbiór w Pythonie – unikalne elementy

1.6 Instrukcje warunkowe i pętle

Instrukcje warunkowe oraz pętle pozwalają tworzyć bardziej złożoną logikę programu. **Instrukcje warunkowe** wybierają fragment kodu do wykonania w zależności od spełnienia warunku logicznego, natomiast **pętle** umożliwiają wielokrotne wykonywanie tego samego fragmentu kodu.

1.6.1 Instrukcje warunkowe if/elif/else

Instrukcja if sprawdza, czy warunek logiczny jest prawdziwy (True). Jeśli tak, wykonywany jest kod w odpowiednim bloku. Dodatkowo można użyć elif (else if) i else.

```
1 x = 5
2
3 if x > 0:
4     print("x jest dodatnie")
5 elif x == 0:
6     print("x jest równe zero")
7 else:
8     print("x jest ujemne")
```

Kod źródłowy 18: Instrukcja warunkowa if/elif/else

Wynik działania programu Kod źródłowy 18 w konsoli:

```
1 x jest dodatnie
```

Kod źródłowy 19: Wynik działania programu w konsoli

1.6.2 Pętla for

Pętla for w Pythonie służy do iteracji po elementach kolekcji (lista, napis, krotka, słownik) albo po zakresie liczb wygenerowanym przez funkcję range. W odróżnieniu od języków C/C++, nie ma tu klasycznej pętli for(i=0; i<n; i++).

```
1 # iteracja po zakresie 0..4
2 for i in range(5):
3     print(i)
4
5 # iteracja po elementach listy
6 animals = ["kot", "pies", "mysz"]
7 for a in animals:
8     print(a)
```

Kod źródłowy 20: Pętla for w Pythonie

Wynik działania programu Kod źródłowy 20 w konsoli:

```
1 0
2 1
3 2
4 3
5 4
6 kot
7 pies
8 mysz
```

Kod źródłowy 21: Wynik działania programu w konsoli

1.6.3 Pętla while

Pętla `while` wykonuje blok kodu dopóki warunek logiczny jest spełniony. Trzeba uważać, aby warunek w pewnym momencie stał się fałszywy, w przeciwnym razie pętla będzie działać w nieskończoność.

```
1 x = 0
2 while x < 5:
3     print(x)
4     x += 1 # zwiększanie wartości x
```

Kod źródłowy 22: Pętla while

Wynik działania programu Kod źródłowy 22 w konsoli:

```
1 0
2 1
3 2
4 3
5 4
```

Kod źródłowy 23: Wynik działania programu w konsoli

1.6.4 Dodatkowe instrukcje w pętlach

Python udostępnia dodatkowe polecenia do kontroli przebiegu pętli:

- `break` – natychmiast kończy działanie pętli,
- `continue` – przerywa bieżącą iterację i przechodzi do następnej,
- `pass` – nic nie robi (przydatne jako „puste miejsce” w kodzie).

```
1 for i in range(10):
2     if i == 3:
3         continue # pomini 3
4     if i == 7:
5         break # zakończ pętlę
6     print(i)
7
8 # użycie pass jako placeholdera
9 for i in range(3):
10    pass
```

Kod źródłowy 24: Instrukcje break, continue i pass

Wynik działania programu Kod źródłowy 24 w konsoli:

```
1 0
2 1
3 2
4 4
5 5
6 6
```

Kod źródłowy 25: Wynik działania programu w konsoli

Funkcja `range(start, stop, step)` generuje kolejne liczby, np. `range(2, 10, 2) → 2, 4, 6, 8`.

1.7 Funkcje

Funkcje pozwalają grupować fragmenty kodu w bloki, które można wielokrotnie wywoływać z różnymi argumentami. Dzięki temu program staje się bardziej czytelny i łatwiejszy w utrzymaniu. Nazwy funkcji zwykle zapisuje się w konwencji `snake_case`, np. `compute_velocity()`.

1.7.1 Definicja funkcji

Funkcje definiuje się słowem kluczowym `def`. Argumenty mogą być obowiązkowe lub opcjonalne (z wartościami domyślnymi).

```
1 def dodaj(a, b=0):    # b ma wartość domyślną 0
2     return a + b
3
4 print(dodaj(5, 2))    # wynik: 7
5 print(dodaj(5))       # wynik: 5 (b przyjęło wartość domyślną)
```

Kod źródłowy 26: Definicja i wywołanie funkcji

1.7.2 Argumenty funkcji

- Argumenty przekazywane są przez referencję (dla typów złożonych, np. lista).
- Funkcje mogą zwracać dowolny obiekt przy pomocy instrukcji `return`.
- Można zwrócić więcej niż jedną wartość w postaci krotki.

```
1 def min_max(values):
2     # Funkcja zwracająca dwie wartości (minimalna i maksymalna) w postaci krotki z podanego zbioru.
3     return min(values), max(values)
4
5 result = min_max([3, 7, 2, 9])
6 print(result)      # (2, 9)
```

Kod źródłowy 27: Funkcja zwracająca więcej niż jedną wartość

1.8 Klasy

Klasy są mechanizmem programowania obiektowego w Pythonie. Pozwalają łączyć dane (atrybuty) i funkcje (metody) w jeden obiekt. Każdy obiekt jest instancją klasy.

1.8.1 Definicja klasy

Klasy definiuje się słowem kluczowym `class`. Konstruktor klasy to metoda specjalna `__init__`, która jest wywoływana przy tworzeniu nowego obiektu. Pierwszy argument metod klasy to zawsze `self`, który odnosi się do bieżącego obiektu.

```
1 class Robot:
2     def __init__(self, name):
3         self.name = name    # atrybut obiektu
4
5     def przedstaw_sie(self): # metoda przedstaw_sie
6         print(f"Jestem {self.name}")
7
8 r = Robot("A1")             # utworzenie obiektu klasy Robot
9 r.przedstaw_sie()          # wywołanie metody; wyświetla tekst: "Jestem A1"
```

Kod źródłowy 28: Definicja klasy i utworzenie obiektu

1.8.2 Atrybuty i metody

- **Atrybuty** – przechowują dane związane z obiektem (np. `self.name`).
- **Metody** – funkcje zdefiniowane wewnątrz klasy, operujące na danych obiektu.

```
1 class Licznik:
2     def __init__(self):
3         self.value = 0
4
5     def increment(self):
6         self.value += 1
7
8 c = Licznik()
9 c.increment()    # wywołanie metody increment, która zwiększa wartość atrybutu value
10 print(c.value)  # wyświetla wynik: 1
```

Kod źródłowy 29: Atrybuty i metody klasy

1.8.3 Dziedziczenie

Klasy mogą dziedziczyć po innych klasach. Pozwala to na ponowne użycie kodu i rozszerzanie funkcjonalności.

```
1 class Pojazd:
2     def __init__(self, nazwa):
3         self.nazwa = nazwa
4     def info(self):
5         print(f"Pojazd: {self.nazwa}")
6
7 class Robot(Pojazd):    # dziedziczenie
8     def info(self):
9         print(f"Robot: {self.nazwa}")
10
11 r = Robot("A1")
12 r.info()               # "Robot: A1"
```

Kod źródłowy 30: Dziedziczenie klas w Pythonie

Uwagi praktyczne

- Konwencja nazewnicza klas to CamelCase, np. `MobileRobot`.
- Python nie posiada modyfikatorów dostępu (`public`, `private`), ale przyjęto konwencję:
 - `_` atrybut – „chroniony”,
 - `__` atrybut – „prywatny”.

2 Operacje matematyczne

2.1 Przydatne funkcje matematyczne

W Pythonie do obliczeń matematycznych używa się głównie dwóch bibliotek:

- **math** – proste operacje matematyczne (funkcje trygonometryczne, potęgi, pierwiastki, liczba π).
- **numpy** (alias `np`) – obliczenia numeryczne i macierzowe. Pozwala w prosty sposób pracować na wektorach i macierzach.

2.1.1 Stałe i podstawowe funkcje

```

1 import math
2 import numpy as np
3
4 print(math.pi)          # 3.14159...
5 print(np.pi)           # 3.14159...
6
7 print(math.e)           # 2.71828...
8 print(np.e)            # 2.71828...
9
10 print(math.sqrt(16))    # 4.0
11 print(np.sqrt(25))     # 5.0
12
13 # konwersje kątów
14 print(math.radians(180)) # 3.14159...
15 print(math.degrees(np.pi)) # 180.0

```

Kod źródłowy 31: Stałe i podstawowe funkcje z bibliotek `math` i `numpy`

2.1.2 Funkcje trygonometryczne

Funkcje `sin`, `cos`, `tan` przyjmują argument kąta w radianach. Do wyznaczania kąta na podstawie wartości trygonometrycznej służą funkcje odwrotne (`asin`, `acos`, `atan`, `arctan2`). Przydatne są także funkcje `math.radians` i `math.degrees` do konwersji jednostek kąta.

```

1 import math
2 import numpy as np
3
4 # math.radians - konwersja stopni na radiany; argumentem funkcji jest kąt w stopniach
5 angle = math.radians(30) # 30° = 0.523 rad
6 print(math.sin(angle))   # 0.5
7 print(math.cos(angle))   # 0.866...
8 print(math.tan(angle))   # 0.577...
9
10 # funkcje odwrotne
11 # math.degrees - konwersja radianów na stopnie; argumentem funkcji jest kąt w radianach
12 print(math.degrees(math.asin(0.5))) # 30.0
13 print(math.degrees(math.acos(0.5))) # 60.0
14 print(math.degrees(math.atan(1)))   # 45.0
15
16 # arctan2(y, x) - kąt punktu (x,y)
17 print(np.arctan2(1, 1))             # 0.785... rad = 45°
18 print(np.arctan2(1, 0))             # 1.571... rad = 90°
19 print(np.arctan2(-1, 0))            # -1.571... rad = -90°

```

Kod źródłowy 32: Funkcje trygonometryczne i odwrotne w Pythonie

2.1.3 Funkcja `np.arctan2`

- Funkcja `np.arctan2(y, x)` wyznacza kąt w radianach pomiędzy osią X a wektorem (x, y) w układzie współrzędnych 2D.
- Wynik mieści się zawsze w zakresie $[-\pi, \pi]$, co oznacza, że funkcja rozróżnia wszystkie cztery ćwiartki układu współrzędnych.

Dlaczego nie wystarczy `atan`?

- Funkcja `atan(y/x)` nie rozróżnia znaku x i y , dlatego nie pozwala określić, w której ćwiartce znajduje się punkt.
- Przykład: punkt $(-1, 1)$ i $(1, -1)$ dają tę samą wartość $y/x = -1$, a więc `atan(-1)` nie odróżnia ich położenia.
- Funkcja `arctan2` eliminuje ten problem, ponieważ przyjmuje dwa argumenty: najpierw y , a potem x .

Podstawowe użycie

```
1 # punkt w I ćwiartce
2 print(np.arctan2(1, 1))      # 0.785 rad = 45°
3
4 # punkt na osi Y
5 print(np.arctan2(1, 0))      # 1.571 rad = 90°
6
7 # punkt w II ćwiartce
8 print(np.arctan2(1, -1))     # 2.356 rad = 135°
9
10 # punkt w IV ćwiartce
11 print(np.arctan2(-1, 1))     # -0.785 rad = -45°
```

Kod źródłowy 33: Podstawowe przykłady użycia `np.arctan2`

Specjalne przypadki

Funkcja działa poprawnie także w szczególnych przypadkach:

```
1 # specjalne przypadki
2 print(np.arctan2(0, 0))      # 0.0
3 print(np.arctan2(1, 0))      # 1.571 rad = 90°
4 print(np.arctan2(-1, 0))     # -1.571 rad = -90°
5 print(np.arctan2(0, -1))     # 3.142 rad = 180°
```

Kod źródłowy 34: Specjalne przypadki działania funkcji `arctan2`

Wektorowe użycie

`np.arctan2` działa elementowo na tablicach NumPy, dzięki czemu w prosty sposób można obliczyć kąty dla wielu punktów jednocześnie.

```
1 x = np.array([-1, 1, 1, -1])
2 y = np.array([-1, -1, 1, 1])
3
4 angles = np.arctan2(y, x) * 180 / np.pi
5 print(angles) # [-135. -45. 45. 135.]
```

Kod źródłowy 35: Użycie `np.arctan2` na tablicach NumPy

2.2 Wektory i macierze

Najczęściej używanym typem danych w `numpy` jest tablica (`ndarray`). Może reprezentować wektory i macierze.

```
1 # wektor
2 v = np.array([1, 2, 3])
3 print(f"{v=}")
4 print(f"Rozmiar wektora: {v.shape}") # rozmiar wektora -> (3, 1)
5
6 # macierz 2x2
7 M = np.array([[1, 2],
8               [3, 4]])
9 print(f"{M=}")
10 print(f"Rozmiar macierzy: {M.shape}") # rozmiar macierzy -> (2, 2)
```

Kod źródłowy 36: Tworzenie wektorów i macierzy

Wynik działania programu Kod źródłowy 36 w konsoli:

```
1 v=array([1, 2, 3])
2 Rozmiar wektora: (3,)
3 M=array([[1, 2],
4          [3, 4]])
5 Rozmiar macierzy: (2, 2)
```

Kod źródłowy 37: Wynik działania programu w konsoli

2.2.1 Operacje na macierzach

Generowanie macierzy i ich transponowanie

```

1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(f"Macierz A - wymiary macierzy {A.shape}: \n {A=} \n")
4 # \n <- znak nowej linii
5 AT = A.T # transpozycja macierzy A
6 print(f"Macierz transponowana AT - wymiary macierzy {AT.shape}: \n {AT=} \n")
7
8 A_np_t = np.transpose(A) # transpozycja macierzy A; działa tak samo jak A.T
9 print(f"Macierz transponowana A_np_t - wymiary macierzy {A_np_t.shape}: \n {A_np_t=} \n")
10
11 D = np.diag([1, 2, 3]) # utworzenie macierzy diagonalnej
12 print(f"Macierz diagonalna D- wymiary macierzy \n {D.shape}: {D=} \n")

```

Kod źródłowy 38: Generowanie macierzy i ich transponowanie

Wynik działania programu Kod źródłowy 38 w konsoli:

```

1 Macierz A - wymiary macierzy (2, 3):
2 A=array([[1, 2, 3],
3          [4, 5, 6]])
4
5 Macierz transponowana AT - wymiary macierzy (3, 2):
6 AT=array([[1, 4],
7           [2, 5],
8           [3, 6]])
9
10 Macierz transponowana A_np_t - wymiary macierzy (3, 2):
11 A_np_t=array([[1, 4],
12              [2, 5],
13              [3, 6]])
14
15 Macierz diagonalna D- wymiary macierzy (3, 3):
16 D=array([[1, 0, 0],
17          [0, 2, 0],
18          [0, 0, 3]])

```

Kod źródłowy 39: Wynik działania programu w konsoli

Mnożenie macierzy (@)

```

1 # Przykład 1
2 A = np.array([[1, 2, 3],
3               [4, 5, 6]])
4
5 B = np.array([[2, 0, 1],
6               [1, 3, 2],
7               [0, 1, 1]])
8 C = A @ B # operator @ = mnożenie macierzowe
9 print(f"Przykład 1 \n {A=} \n {B=} \n")
10 print(f"Wymiary macierzy: A={A.shape}, B={B.shape}, C={C.shape} \n {C=} \n")
11 print("-----\n")
12
13 # Przykład 2
14 M1 = np.array([[1,0,0],
15                [0,2,0],
16                [0,0,3]])
17 M2 = np.array([[1,2,3],
18                [3,4,5],
19                [1,1,2]])
20 W = M1 @ M2 # operator mnożenia macierzy
21 print(f"Przykład 2 \n {M1=} \n {M2=} \n")
22 print(f"Wymiary macierzy: M1={M1.shape}, M2={M2.shape}, W={W.shape} \n {W=} \n")

```

Kod źródłowy 40: Mnożenie macierzy @

Wynik działania programu Kod źródłowy 40 w konsoli:

```

1 Przykład 1
2 A=array([[1, 2, 3],
3          [4, 5, 6]])
4 B=array([[2, 0, 1],
5          [1, 3, 2],
6          [0, 1, 1]])
7
8 Wymiary macierzy: A=(2, 3), B=(3, 3), C=(2, 3)

```

```

9  C=array([[ 4,  9,  8],
10         [13, 21, 20]])
11
12  -----
13
14  Przykład 2
15  M1=array([[1, 0, 0],
16           [0, 2, 0],
17           [0, 0, 3]])
18  M2=array([[1, 2, 3],
19           [3, 4, 5],
20           [1, 1, 2]])
21
22  Wymiary macierzy: M1=(3, 3), M2=(3, 3), W=(3, 3)
23  W=array([[ 1,  2,  3],
24          [ 6,  8, 10],
25          [ 3,  3,  6]])

```

Kod źródłowy 41: Wynik działania programu w konsoli

Funkcja np.dot

- Funkcja `np.dot` służy do wykonywania iloczynu skalarnego lub mnożenia macierzy, w zależności od wymiaru podanych argumentów.
- Dla **wektorów 1D** (`np.array([a,b,c])`) – oblicza **iloczyn skalarny** (sumę iloczynów elementów).
- Dla **macierzy 2D** – wykonuje **mnożenie macierzy** zgodne z algebrą liniową.
- Dla **wektora i macierzy** – zachowuje się jak klasyczne mnożenie w algebrze liniowej.

```

1  # --- iloczyn skalarny ---
2  a = np.array([1, 2, 3])
3  b = np.array([4, 5, 6])
4  print(np.dot(a, b))    # 1*4 + 2*5 + 3*6 = 32
5
6  # --- mnożenie macierzy ---
7  A = np.array([[1, 2],
8               [3, 4]])
9  B = np.array([[5, 6],
10              [7, 8]])
11 print(np.dot(A, B))
12 # [[19 22]
13 #  [43 50]]
14
15 # --- macierz i wektor ---
16 v = np.array([1, 2])
17 print(np.dot(A, v))    # [1*1 + 2*2, 3*1 + 4*2] = [5, 11]

```

Kod źródłowy 42: Przykłady działania funkcji `np.dot`

2.2.2 Konwersja danych NumPy na listy Pythona

Tablice (`ndarray`) w bibliotece NumPy są bardzo wydajne w obliczeniach matematycznych, ale czasem konieczne jest przekształcenie ich do zwykłych list Pythona, np. aby zapisać dane w pliku JSON, przesłać przez ROS 2 albo przekazać do funkcji, która nie obsługuje typów NumPy.

Do konwersji służy metoda `.tolist()`, która działa zarówno na wektorach (1D), jak i na macierzach (2D i wyższych wymiarów). Metoda `tolist()` działa rekurencyjnie, więc dla tablic wielowymiarowych tworzy odpowiednio zagnieżdżone listy.

Przykład – konwersja wektora

```

1  # wektor NumPy (1D)
2  v = np.array([1, 2, 3])
3  print(type(v))    # <class 'numpy.ndarray'>
4
5  # konwersja do listy
6  v_list = v.tolist()

```

```

7 print(type(v_list)) # <class 'list'>
8 print(v_list)      # [1, 2, 3]

```

Kod źródłowy 43: Konwersja wektora NumPy do listy Pythona

Przykład – konwersja macierzy

```

1 # macierz NumPy (2D)
2 B = np.array([[1,0,0],
3 [0,2,0],
4 [0,0,3]])
5 print(f"Typ danych B: {type(B)}")
6 print(B)
7
8 # konwersja do listy zagnieżdżonej
9 B_list = B.tolist()
10 print(f"Typ danych B_list: {type(B_list)}")
11 print(B_list)

```

Kod źródłowy 44: Konwersja macierzy NumPy do listy list w Pythonie

Wynik działania programu Kod źródłowy 44 w konsoli:

```

1 Typ danych B: <class 'numpy.ndarray'>
2 [[1 0 0]
3  [0 2 0]
4  [0 0 3]]
5 Typ danych B_list: <class 'list'>
6 [[1, 0, 0], [0, 2, 0], [0, 0, 3]]

```

Kod źródłowy 45: Wynik działania programu w konsoli

Konwersja listy Pythona do tablicy NumPy

Odwrotna operacja jest równie prosta – do stworzenia tablicy NumPy z listy (jedno- lub wielowymiarowej) używa się funkcji `np.array()`. Dzięki NumPy można potem wykonywać operacje macierzowe (`@`, `np.linalg.inv`, `np.dot` itp.), których zwykłe listy nie obsługują.

```

1 # lista 1D w Pythonie
2 v_list = [1, 2, 3]
3
4 # konwersja do tablicy NumPy
5 v = np.array(v_list)
6 print(type(v_list)) # <class 'list'>
7 print(type(v))      # <class 'numpy.ndarray'>
8 print(v)            # [1 2 3]
9
10 # lista 2D (lista list)
11 B_list = [[1,0,0],
12 [0,2,0],
13 [0,0,3]]
14
15 # konwersja do macierzy NumPy
16 B = np.array(B_list)
17 print(type(B))      # <class 'numpy.ndarray'>
18 print(B)

```

Kod źródłowy 46: Konwersja list Pythona do tablic NumPy

2.2.3 Macierz odwrotna i pseudoinwersja

W algebrze liniowej macierz odwrotna pełni podobną rolę jak liczba odwrotna w arytmetyce. Dla macierzy A jej odwrotność A^{-1} spełnia warunek:

$$A \cdot A^{-1} = I$$

gdzie I to macierz jednostkowa.

Nie każda macierz ma odwrotność – warunkiem koniecznym jest, aby macierz była:

- kwadratowa ($n \times n$),
- o niezerowym wyznaczniku (macierz *odwracalna*).

W praktyce jednak często spotykamy macierze prostokątne (np. układy równań nadokreślonych lub nie-dookreślonych). Wtedy używa się **pseudoinwersji Moore’a–Penrose’a**, która działa również dla macierzy niekwadratowych i przybliża rozwiązania w sensie najmniejszych kwadratów.

Funkcje w NumPy

- `np.linalg.inv(A)` – oblicza macierz odwrotną (tylko dla macierzy kwadratowych i odwracalnych),
- `np.linalg.pinv(A)` – oblicza pseudoinwersję **Moore’a–Penrose’a** (działa dla dowolnych macierzy, także prostokątnych i osobliwych).

Przykład – macierz odwrotna

```
1 C = np.array([[1, 2],
2 [3, 4]])
3
4 # macierz odwrotna
5 Cinv = np.linalg.inv(C)
6 print("C^-1 =\n", Cinv)
```

Kod źródłowy 47: Wyznaczanie macierzy odwrotnej

Przykład – pseudoinwersja

```
1 # macierz prostokątna (3x2)
2 D = np.array([[1, 0],
3 [0, 2],
4 [0, 3]])
5
6 # pseudoinwersja Moore'a-Penrose'a
7 Dpinv = np.linalg.pinv(D)
8 print("D^+ =\n", Dpinv)
```

Kod źródłowy 48: Pseudoinwersja macierzy prostokątnej

Uwagi praktyczne

- `np.linalg.inv` zwróci błąd, jeśli macierz nie jest odwracalna (wyznacznik = 0).
- `np.linalg.pinv` zawsze działa – korzysta z rozkładu SVD (*ang. Singular Value Decomposition*).
- Pseudoinwersja jest często stosowana w:
 - rozwiązywaniu układów równań nadokreślonych (metoda najmniejszych kwadratów),
 - kinematyce odwrotnej w robotyce.

3 ROS 2 i Python

Python jest językiem, w którym programy w ROS2 (tzw. nody) definiuje się w postaci klas i funkcji. Niższa część zawiera ogólny opis użycia Pythona w kontekście ROS 2, w szczególności w odniesieniu do składni wykorzystanej w szablonach ćwiczeń. Szczegółowe omówienie kodu oraz jego działania znajduje się w dedyko-

wanych instrukcjach laboratoryjnych dla robota. Tutaj celem jest przede wszystkim zaprezentowanie przykładu użycia, a nie szczegółowa analiza i wyjaśnienie implementacji.

Podstawowe elementy kodu to:

- **importy** – wczytanie bibliotek Pythona i ROS 2,
- **klasa noda** – definiuje logikę działania (dziedziczenie po **Node**),
- **callbacki** – metody wywoływane automatycznie po odebraniu wiadomości,
- **publisher/subscriber** – obiekty pozwalające wysyłać i odbierać dane,
- **funkcja main** – uruchamia cały program i obsługuje pętlę zdarzeń.

3.1 Przykładowe importy w Pythonie dla ROS 2

- `import rclpy` – podstawowa biblioteka ROS 2 dla Pythona,
- `from rclpy.node import Node` – klasa bazowa, po której dziedziczą wszystkie nasze nody,
- `from sensor_msgs.msg import JointState` – importuje wiadomość dla tematów (bo `.msg`) typu **JointState** z pakietu **sensor_msgs**. **JointState** - struktura danych używana do zapisu stanu kół (prędkości, pozycje),
- `from geometry_msgs.msg import Twist` – importuje wiadomość dla tematów typu **Twist** z pakietu **geometry_msgs**. **Twist** - struktura opisująca prędkości liniowe i kątowe,
- `from nav_msgs.msg import Odometry` – importuje wiadomość dla tematów typu **Odometry** z pakietu **nav_msgs**. **Odometry** - struktura opisująca pozycję i orientację robota (odometria).

3.2 Definiowanie noda w Pythonie

Każdy node w ROS 2 definiujemy jako klasę dziedziczącą po **Node**. W konstruktorze `__init__` tworzymy subskrypcje i publishery.

```
1 class Robot(Node):
2     def __init__(self):
3         # wywołanie konstruktora klasy bazowej
4         super().__init__('robot_inverse_kinematics')
5
6         # subskrypcja tematu /cmd_vel
7         self.subscription_cmd_vel = self.create_subscription(
8             Twist,
9             '/cmd_vel',
10            self.cmd_vel_callback,
11            10
12        )
13
14        # publisher na temat /cmd_joint_states
15        self.publisher_joint_states = self.create_publisher(
16            JointState,
17            '/cmd_joint_states',
18            10
19        )
```

Kod źródłowy 49: Definicja klasy noda w Pythonie

Wyjaśnienie składni Pythona

- `class Robot(Node):` – definicja klasy **Robot**, dziedziczącej po **Node**.
- `def __init__(self):` – konstruktor klasy, uruchamiany przy tworzeniu obiektu.
- `super().__init__('...')` – wywołanie konstruktora klasy nadrzędnej z nazwą noda. Powinna być ona unikalna w odniesieniu do innych już uruchomionych węzłów.

- `self.subscription_cmd_vel = ...` – utworzenie subskrypcji i zapisanie jej jako atrybut klasy.
- `self.publisher_joint_states = ...` – utworzenie publishera.

3.3 Callbacki – funkcje wywoływane automatycznie

Callback to zwykła funkcja Pythona, która przyjmuje jako argument wiadomość odebraną z tematu. Musi mieć parametr `self` (bo w tym przypadku używamy metody klasy `Robot`) oraz parametr `msg` (dane wiadomości). Zamiast `msg` można użyć dowolnej nazwy sugerującej, że jest to odebrana wiadomość przez Subscribiera.

```

1 def cmd_vel_callback(self, msg):
2     """
3     Callback odbierający wiadomość typu Twist.
4     Do uzupełnienia przez studentów: obliczenie prędkości kół
5     i utworzenie wiadomości JointState.
6     """
7     joint_state = JointState()
8     # joint_state.velocity = [...]
9     # joint_state.position = [...]
10    self.publisher_joint_states.publish(joint_state)

```

Kod źródłowy 50: Przykładowy callback w Pythonie, który wysyła również wiadomość typu `JointState`

Analogicznie dla kinematyki prostej używamy callbacku `joint_states_callback`, który odbiera wiadomości typu `JointState` i na ich podstawie publikuje `Odometry`.

3.4 Konwersja kąta na kwaternion

Python pozwala tworzyć funkcje pomocnicze poza klasą. Tutaj przygotowana jest funkcja do przekształcenia kąta `theta` (jedna liczba) w kwaternion (cztery liczby), ponieważ ROS 2 wymaga zawsze kwaternionów:

```

1 def theta_to_quaternion(theta):
2     q = Quaternion()
3     q.z = np.sin(theta / 2.0)
4     q.w = np.cos(theta / 2.0)
5     return q

```

Kod źródłowy 51: Funkcja pomocnicza do wyznaczenia kwaterniona w Pythonie

3.5 Funkcja main w Pythonie

Każdy program w Pythonie dla ROS 2 kończy się funkcją `main`. Jej zadaniem jest uruchomienie systemu, utworzenie noda, włączenie pętli obsługującej zdarzenia i zakończenie pracy.

```

1 def main(args=None):
2     rclpy.init(args=args)          # inicjalizacja ROS2
3     node = Robot()                 # utworzenie noda
4     try:
5         rclpy.spin(node)           # obsługa zdarzeń (callbacków)
6     finally:
7         if rclpy.ok():              # bezpieczne zamknięcie
8             rclpy.shutdown()
9
10    if __name__ == '__main__':
11        main()

```

Kod źródłowy 52: Funkcja główna programu ROS 2 w Pythonie